# AgentCgroup

## AI Agent Resource Characterization and OS-Level Control

Yusheng Zheng (yzhen165@ucsc.edu)
PhD Student

# Roadmap

- Motivation & Problem
- Resource Characterization (144 tasks, 2 models)
- Agent vs. Cloud Workloads & Three Mismatches
- AgentCgroup: eBPF + Bash Wrapper + Negotiation
- Evaluation & Takeaways

# Motivation

AI coding agents **Claude Code, Codex** are being deployed at scale, autonomously running compilers, test suites, and package managers in sandboxed containers.

## New Workloads: AI Agents

- LLM reasons → calls tools → observes → iterates
- Agent **autonomously decides** what to run and when
- Peform actions 24/7, tools may be heavy and spike

## …How much does it actually cost?

- People Host **50–100+ agents per machine**
- Does traditional **containers** and **VMs** enough?
- Most study around Safety, what about **resource usage** and **performance**?
- How can we utilize resources patterns to achieve higher deployment density

# Characterization: Setup

**144 SE tasks** from SWE-rebench benchmark, with Claude Code

| Config | Model | Tasks |
|---|---|---|
| Local GPU | GLM-4.7-Flash | 111 |
| Cloud API | Claude Haiku 4.5 | 33 |

**Hardware**: Intel Core Ultra 9 285K, 24 cores, 128 GB DDR5

**Environment**: Podman containers, Linux 6.15, 1s sampling

## What We Measure

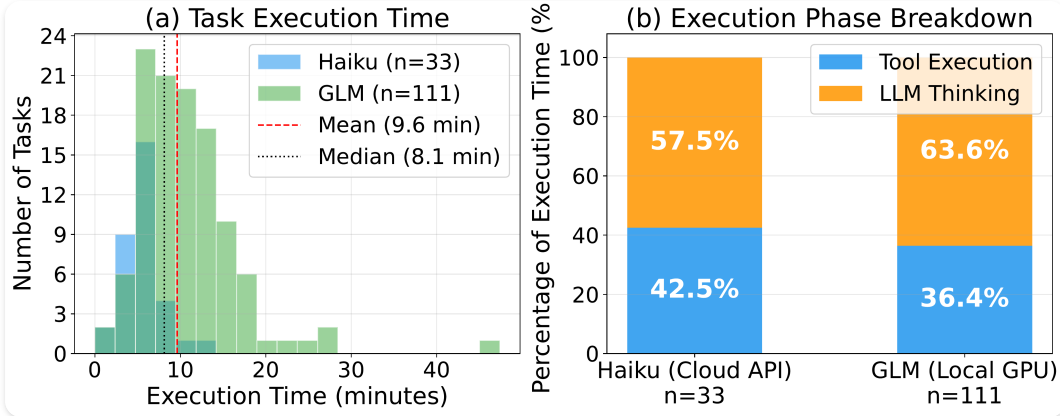
- **Where does agent do?** (execution model)
- **What does resource usage look like?** (resource dynamics)
- **Can demands be predicted?** (cross-task variance)

## Container Images

- **Average 3.5 GB, 7× larger than microservice and 70× larger than FAAS**

# Execution Model

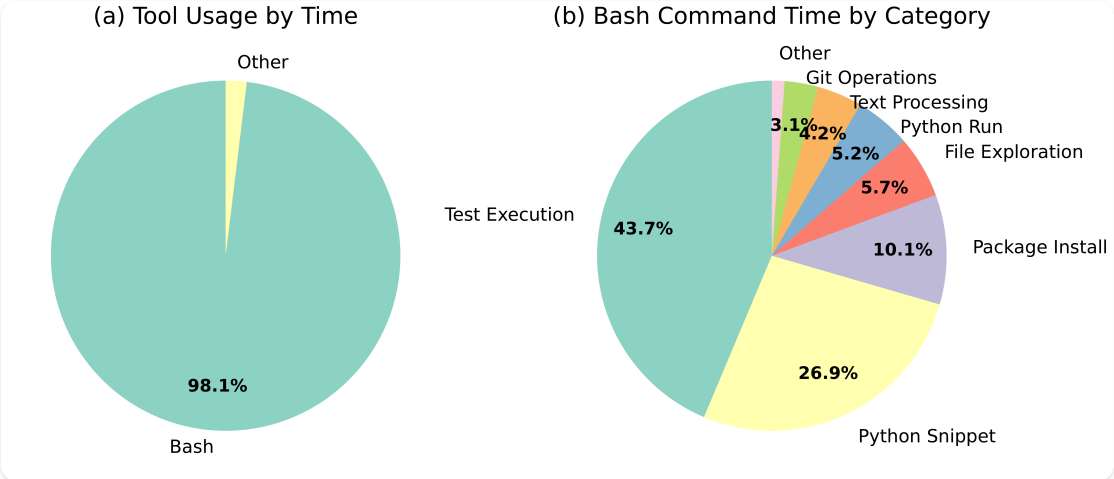
Q: Where does the time go?




- Task duration: mean 9.6 min
- LLM reasoning: 26–44% of end-to-end task latency
- the remainder:
  - tool execution (~40% of active time)
  - setup env and initialization (29–45%)

# What Tool Dominates?

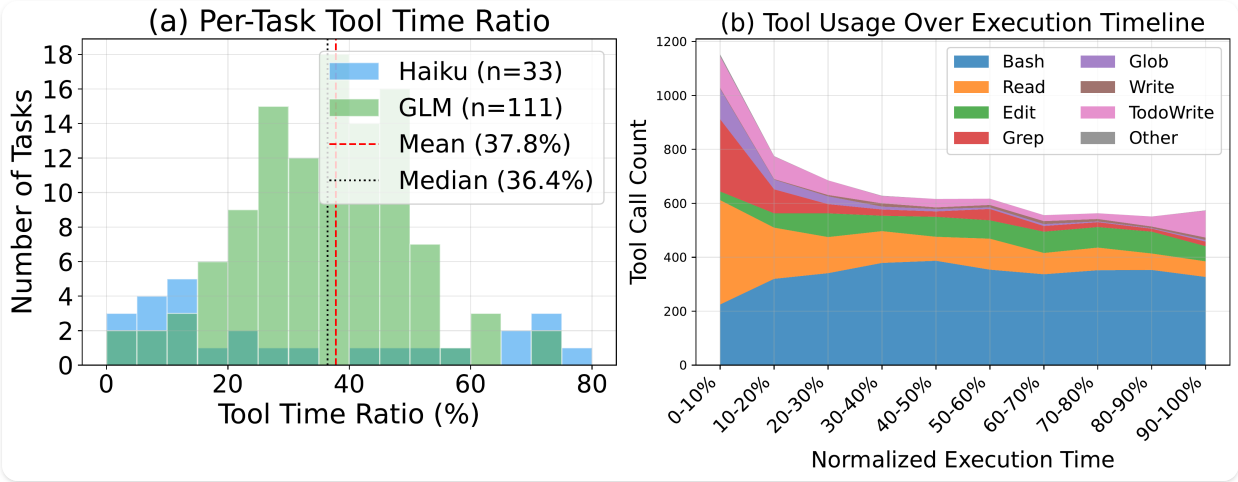
Q: Which tool types consume the most execution time?




- Bash dominates tool execution
- Top bash categories: test execution, Python snippet, package install

# Tool Execution: Temporal Pattern

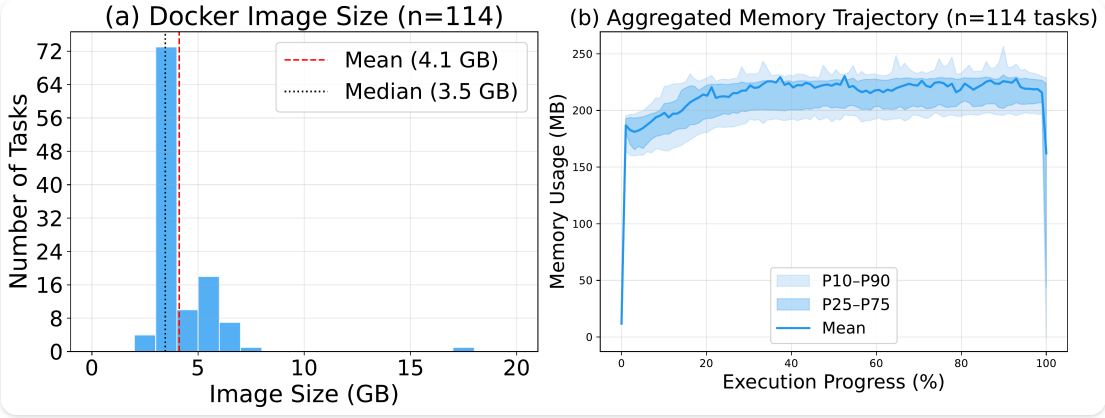
Q: When do different tools execute during a task?




- Tool calls follow an "**understand, modify, verify**" pattern: Read in first 30%, Bash peaks at 40–80%

# Memory & Resource Structure

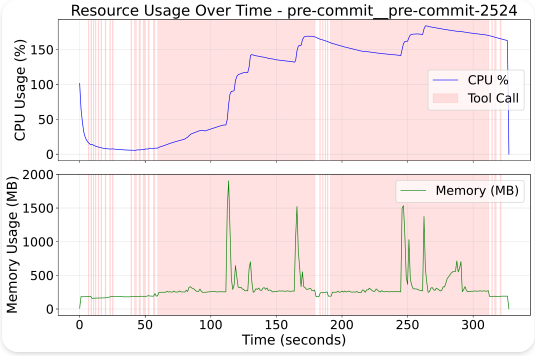
Q: What does resource consumption look like?

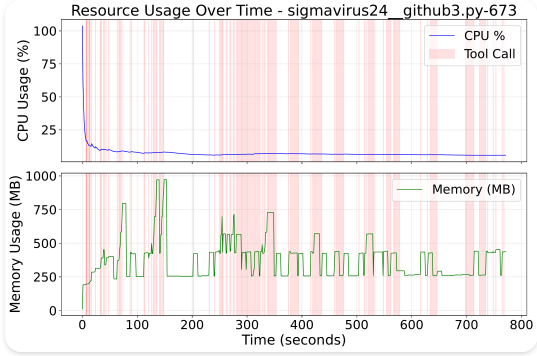



- **Memory, not CPU**, is the primary bottleneck (CPU avg <13%, memory peaks 2–4 GB)
- Resource consumption: **a ~185 MB framework baseline plus tool-call bursts**

# Resource Time Series

## Q: What drives resource bursts?

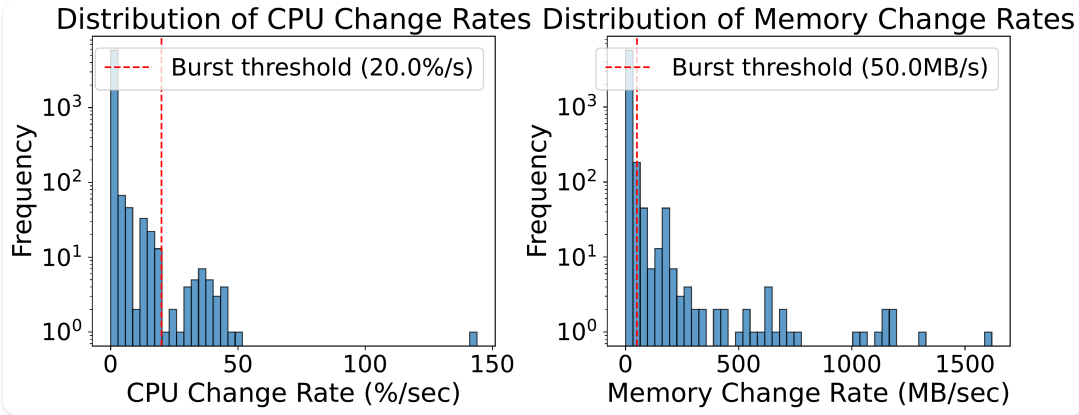



- Resource consumption is determined by what the tool *does* (e.g., pytest vs. git status), not which tool is invoked
  - **Bash calls differ by 13.7× in peak memory**
- **98.5% of memory bursts** occurring during **tool calls**

# Burst Dynamics & Variability

Q: How fast and how variable are resource changes?




- Resource bursts last **1–2 seconds**
- peak-to-average ratio up to **15.4×**
- CPU-memory correlation varies by task **(−0.84 to +0.50); co-directional change cannot be assumed**

# Characterization Summary

1. **OS execution dominates**: 56–74% of latency is tool execution + initialization, not LLM
2. **Memory is the bottleneck**: CPU avg <13%, but memory peaks 2–4 GB
3. **Demands are unpredictable**: 20× across tasks, 1.8× across runs; CPU/memory are decoupled

- **System optimization** is important as LLMs gets faster!
- resource management for AI agents must operate at **tool-call granularity**

# Agent vs. Cloud Workloads

## Q: How different from existing cloud workloads?

| Dimension | Serverless/FaaS | Microservices | Batch/HPC | **AI Coding Agent** |
|---|---|---|---|---|
| Execution duration | 100ms–2s | Long-running | Minutes–hours | **5–11 minutes** |
| Container image | ~50 MB | 100 MB–1 GB | 1–10 GB | **2.9–17.3 GB (med. 3.5)** |
| Memory footprint and peak/avg | 128–512 MB, ~1.5× | Steady ~1 GB, 2–3× | Scales with data | **185 MB idle, peaks 2–4 GB, 15.4×** |
| CPU utilization | Brief spike | 10–40% | 80–100% | **<13% avg, peaks >1.7x** |
| Determinism | Deterministic | Mostly deterministic | Deterministic | **1.8× in same task** |
| Termination cost | Just retry | Can migrate | Lose progress | **Lose progress** |

# Three Mismatches

## Q: Why do existing resource management tools fail?

| | Static Limits | Reactive Control | Predictive Scaling |
|---|---|---|---|
| **Tools** | mem.max/high, cpu.max; K8s QoS | PSI; oomd; TMO | VPA; Autopilot |
| **Assumes** | Known peak; stable demand | Gradual pressure; kill OK | Repeatable; history valid |
| **Agent** | 15.4× peak/avg; tool-semantic | 1–2s burst; unpredictable | 1.8× variance; kill = lose context |
| **Mismatch** | **Granularity** | **Responsiveness** | **Adaptability** |

- **Granularity**: Not fine-grained, `memory.max` at peak wastes >90%; at average → OOM
- **Responsiveness**: Not fast enough, 3 GB/s bursts too fast for user-space tools (tens of ms reaction)
- **Adaptability**: Not semantic understanding, don't know what agent is doing

# Agentcgroup Design and Prelimary Eval

## Design

- Fine-Grained Resource Domains to Seperate the framework from tool call(Bash wrap);
- Using eBPF for in-kernel enforce(Cgroup hooks);
- Intent-Driven Resource Allocation: agent declares what they need via config

## Replay trace to evalution：

- Allow multi-tenant agents less OOM
- Programblely throttle → freeze
- Overhead negligible

# Limitations & Future Work

## Limitations

- Requires patched kernel for prototype ( `memcg_bpf_ops` under review)
- Limited Evaluated
- Agents and tools on one machine

## Future Work

- A LLM inside OS: let it understand what the agent does and negotate with workload
- Seperate the tool env and agent framework into different machine (like OpenClaw)
- Eval on more workoad

# Thank You

Questions?

**References**

AgentCgroup Paper (arXiv:2602.09345) · AgentCgroup GitHub